Embedded Data Structure
(Lecture Note 22)

Searching Algorithms (5)

2025. 5. 28

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class (1/2)

- **Binary tree search**
  - **It is useful for managing your data (cf. binary search)**
  - **Remember rules to traverse a given binary tree for searching**
  - **In the last class, we studied about making a tree**
    - Make a tree by insertion
    - Delete a node according to various cases

    ➔ **We are ready to search a key value in a binary tree**

Review of the Last Class (2/2)

- **C implementation of improved version**
  - **Reduce the number of conditions (i.e., if)**

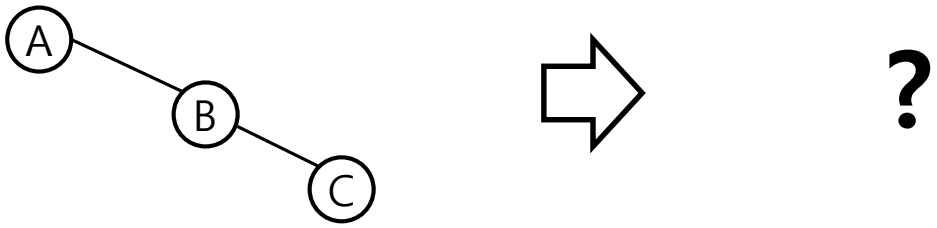
```
node *bti_delete(int key, node *base, int *num)
{
    /* same with bti_delete1()  for finding del node */
    if(del->right == NULL) son = del->left;
    else if(del->right->left == NULL){
        son = del->right;
        son->left = del->left;
    }
    else{
        nexth = del->right;
        while(nexth->left->left != NULL) nexth = nexth->left;
        son = nexth->left;
        netxh->left = son->right;
        son->left = del->left;
        son->right = del->right;
    }
    /* same with bti_delete1()  for making connections with new parent */
}
```

Binary Tree Search (1/6)

▪ Characteristics of binary tree search

- **Efficiency is strongly dependent on the structure of tree**
  - Best case : search length → $\log_2 N$ (binary search)
  - Worst case : search length → N (sequential search)

  ➔ We need to arrange a given tree balanced !

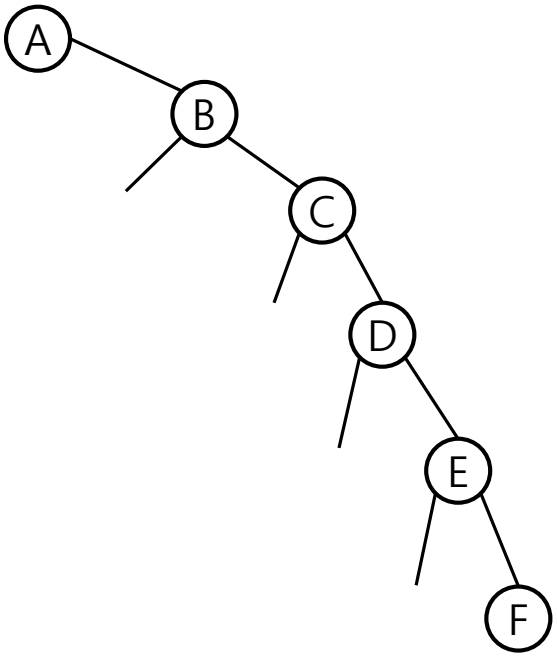


- **Main advantage of binary search : efficiency for dynamic data**
  - It does not require "copy" (many amount of data)

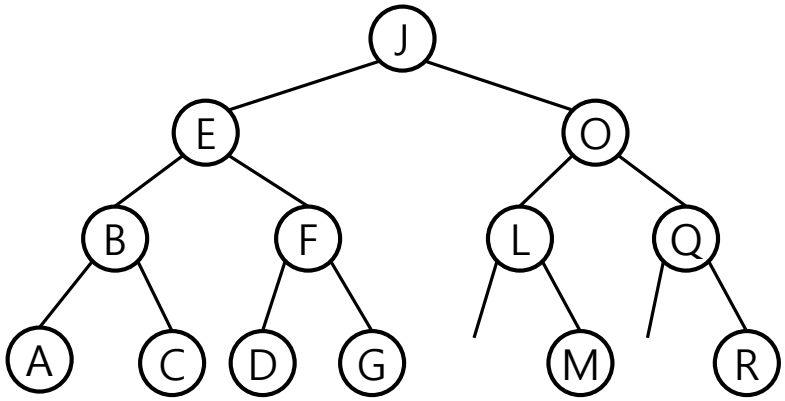
    → We only control the "link" to insert or delete

Binary Tree Search (2/6)

- **Unbalanced vs. balanced**
  - **Efficiency : N vs. $\log_2 N$**



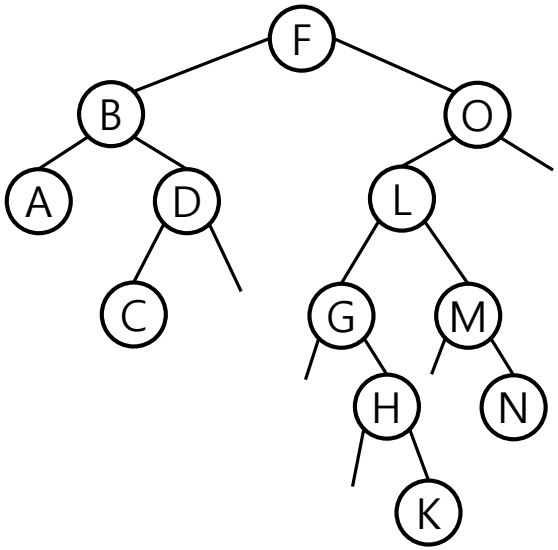
[Unbalanced tree]

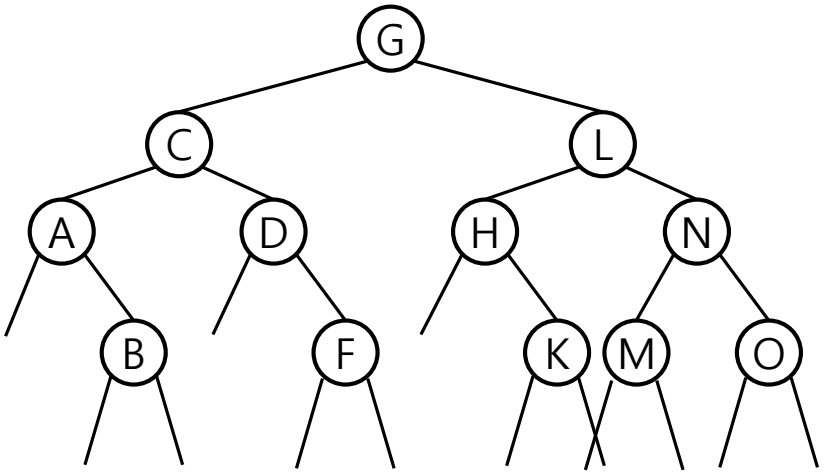


[Balanced tree]

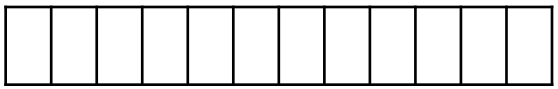
Binary Tree Search (3/6)

- **Tree traverse (for sorting)**
  - **Inorder traverse is employed (why ?)**





[Complete binary tree]



[Result of inorder traverse]

Binary Tree Search (4/6)

- **Sorting of binary tree to be balanced**
  - **Once again, we employ the inorder traverse**

```
void bti_sort(node *p, int *a, int *index)
{
    if(p != NULL)
    {
        bti_sort(p->left, a, index);
        a[            ] = p->key;
        bti_sort(p->right, a, index);
    }
}
```

➡ **Why do we need to use pointer ?**

➡ **What does "a" mean ?**

Balanced tree :

1. N > 0,
    1. (L) and (R) intervals
    2. Node *p
    3. p->left = f(nL);
    4. p->key = array(i);
    5. p->right = f(nR);
    6. Return p;

➡ **Why inorder traverse ?**

Binary Tree Search (5/6)

- **C implementation of balanced tree**
  - **We employ the inorder traverse**

```
node *_balance(int N, int *a, int *index)
{
    int nl, nr;
    node *p;
    if(N > 0){
        nl = (N-1)/2;
        nr = N – nl – 1;
        p = (node *)malloc(sizeof(node));
        p->left = _balance(nl, a, index);
        p->key = a[(*index)++];
        p->right = _balance(nr, a, index);

        return p;
    }
    else return NULL;
}
```

To make a balanced tree,

1) Sort a given tree

2) Conduct _balance ()

Binary Tree Search (6/6)

- **Practical example**
  - **Sort the previous tree (see last note) and make a balanced version**
  - **Conduct the binary tree search !**

Make your code complete !

Summary

- **Make a tree balanced !**
  - **Do your best to completely implement the BTS algorithm**
  - **Try to understand the scheme and related source codes !**